

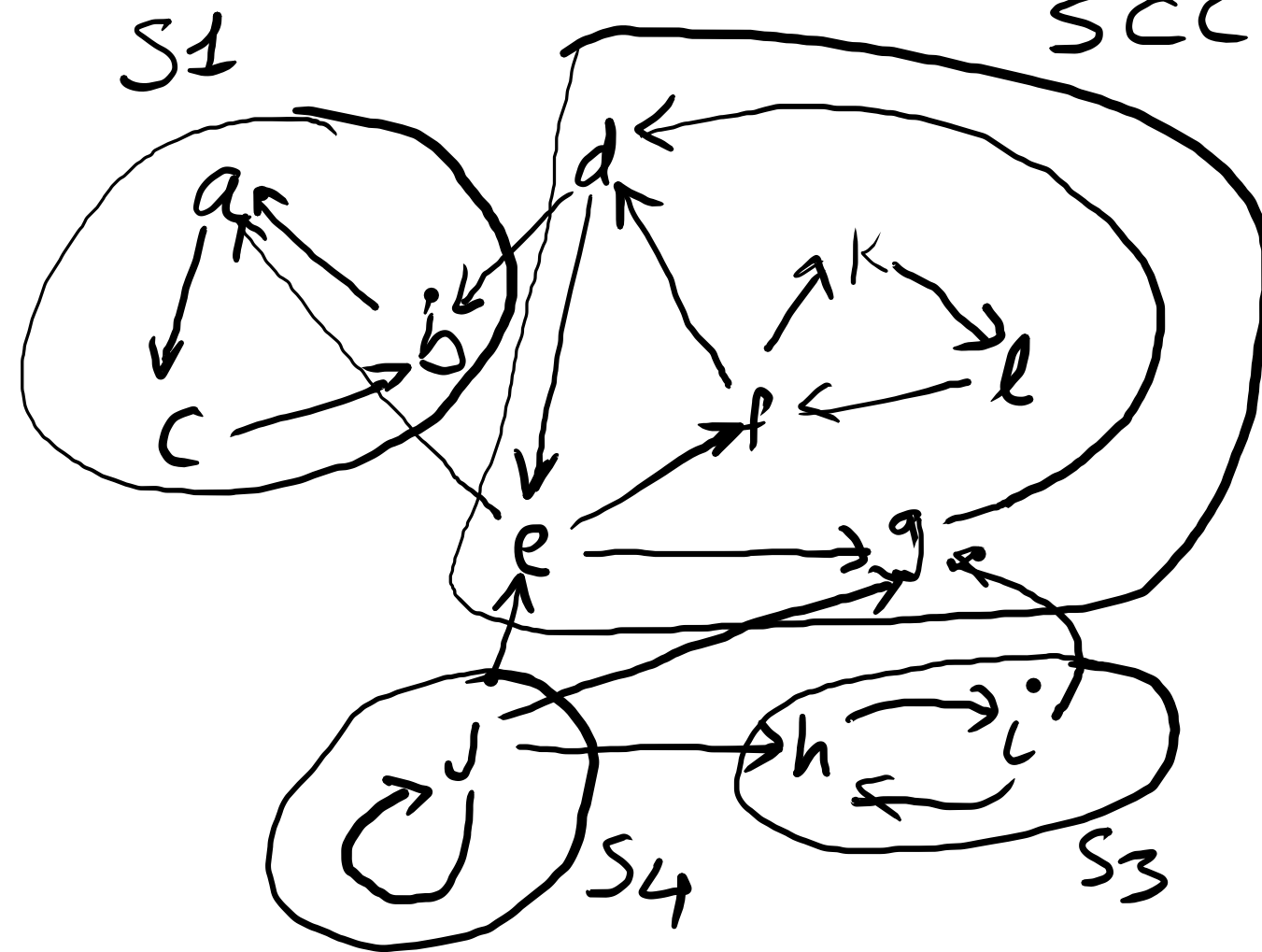
Strongly Connected Components (SCC)

Finding SCCs is done by running DFS twice

Define SCC:

- Let $G(V, E)$ be a directed graph
- SCC of G is a subset S of V such that:
 - For any two vertices $u, v \in S$ it must hold true
 - There is a path from u to v
 - There is a path from v to u
- S is minimal in the sense that we can't put any more vertex in S without violating the above property

SCC Example



S_2 $\{a, b, c\}$ is an SCC

$\{a, b, c, e\}$ is not an SCC
because of e

from e there is a path to
 a or b or c but not from
 a or b or c to e

$\{d, e, f, k, l\}$ is not an SCC because we can add g

$\{d, e, f, k, l, g\} \rightarrow \text{SCC}$

Applications of SCC

- Software module dependency
- Compiler optimization
- Deadlock detection (OS) - using wait-for graphs
- Network analysis and routing
- Biology genetics
- Social Media analysis

SCCs are disjoint

$$S_1 \cap S_2 = \emptyset$$

S_1 and S_2 are SCC

Proof:

Assume vertex v is in both S_1 and S_2

Then for any vertex $u_1 \in S_1$ and $u_2 \in S_2$

We can go from $u_1 \rightarrow v \rightarrow u_2$ and

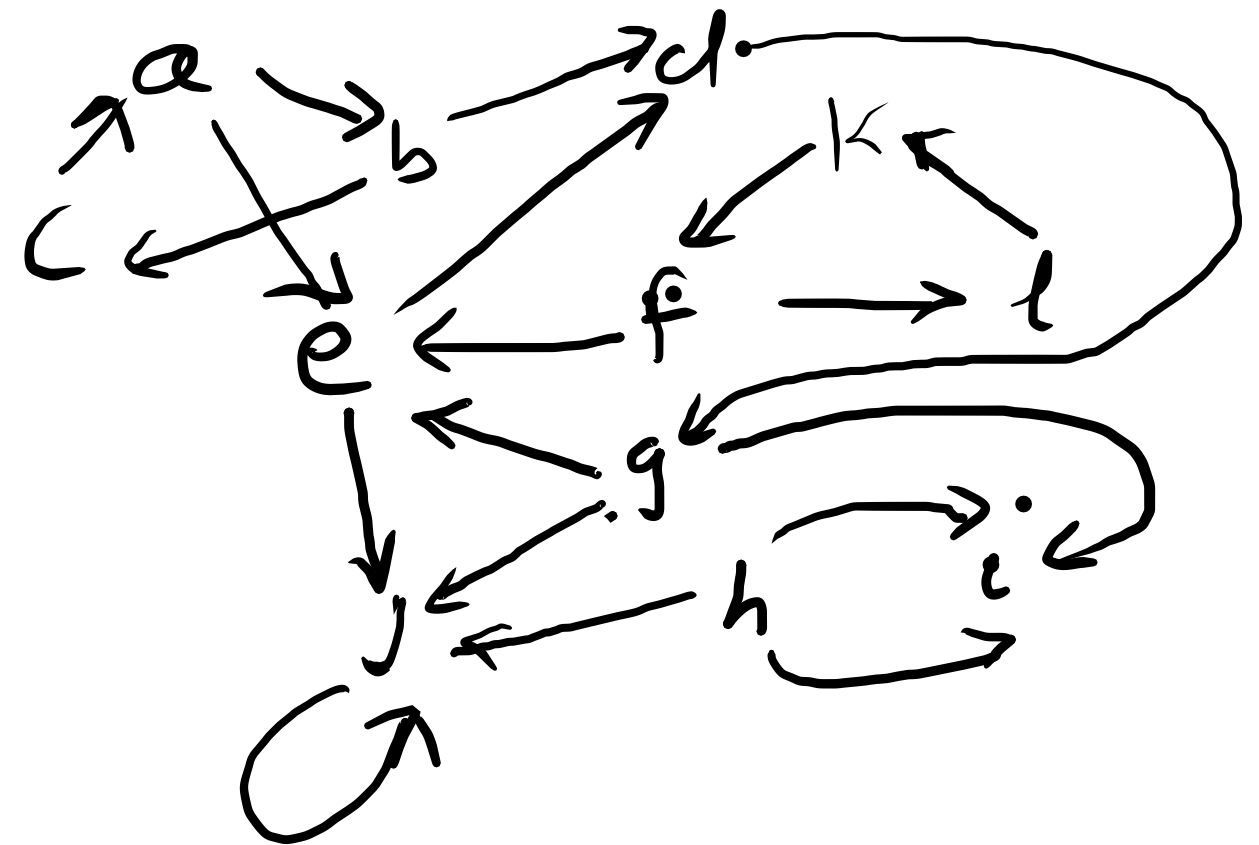
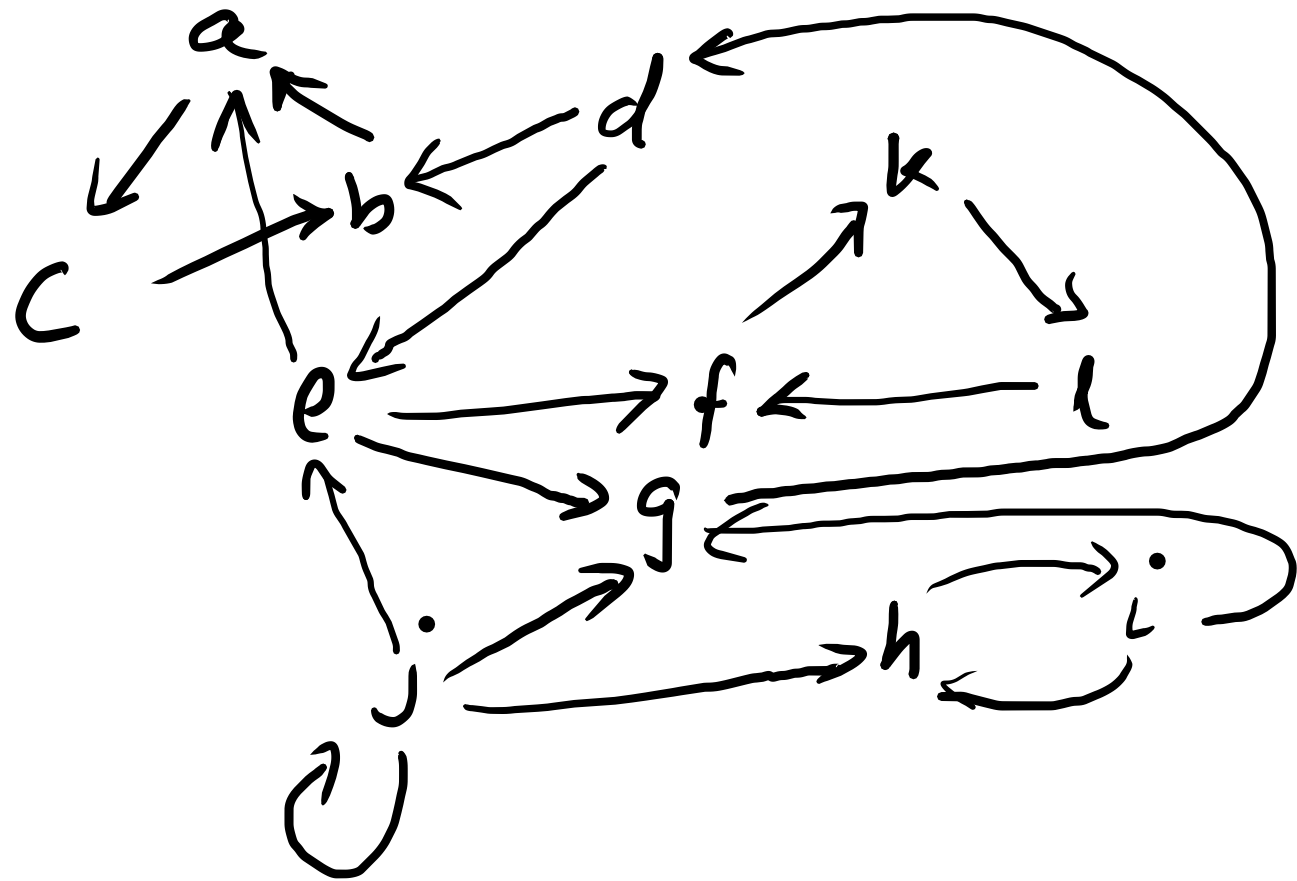
$$u_2 \rightarrow v \rightarrow u_1$$

Hence S_1 and S_2 are not maximal.

Problem of Finding SCC:

Given directed graph $G(V, E)$ the goal of finding SCC problem is to divide G into disjoint subsets each of which is an SCC.

Step 1: Obtain the reverse graph G^R by reversing the directions of all edges in G .

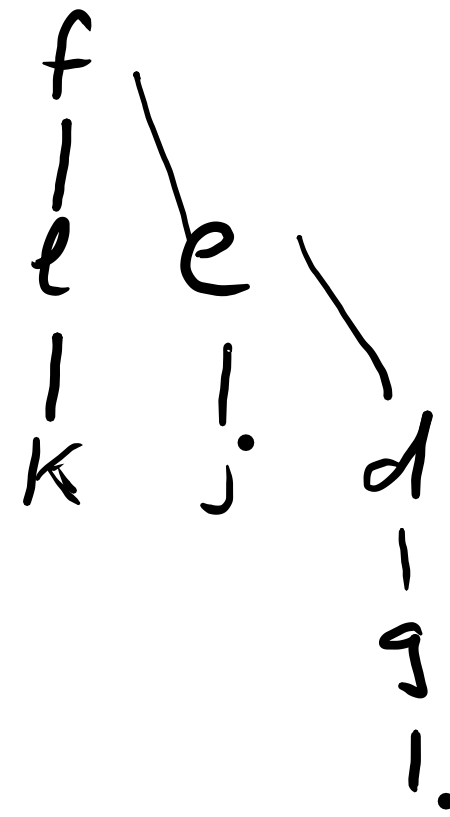
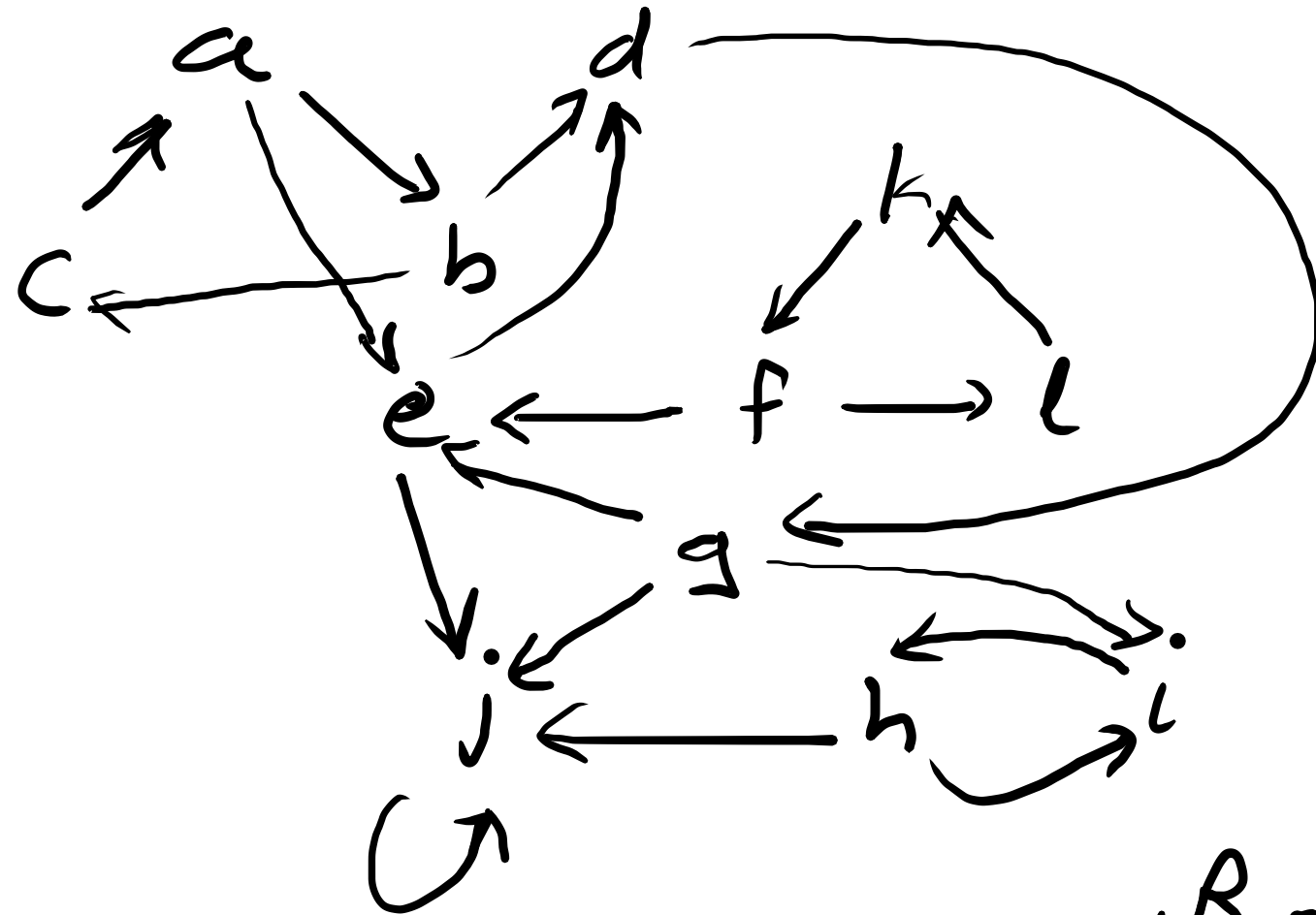


Step 2: - perform DFS on GR and obtain sequence L^R that the vertices in GR turn black

- obtain L as the reverse order of L^R

Run DFS from f

from a



$L^R = \{ k, l, i, h, i, h, g, d, e, f, c, b, a \}$

$L = \{ a, b, c, f, e, d, g, i, h, j, l, k \}$